

Homework 5: Vision Transformers for Image Classification

Gilberto Feliu
Student ID: 801257813

July 20, 2026

Assignment: Homework 5

Course: ECGR 4106

GitHub Repository: <https://github.com/gfeliu-rgb/ECGR-4106-HW5>

Late Submission Note

This homework was submitted late because I had a family medical emergency. My grandmother has pancreatic cancer and was hospitalized during the week of the deadline, which disrupted the time I had planned for completing and packaging the assignment.

Dataset and Experimental Setup

Both problems use CIFAR-100 through `torchvision.datasets.CIFAR100`. Training images are normalized with CIFAR-100 channel statistics. Problem 1 uses 32 by 32 inputs with random crop, padding, and horizontal flip for training; evaluation uses only tensor conversion and normalization. Problem 2 uses 224 by 224 resized inputs for Swin-Tiny, Swin-Small, and the scratch Swin baseline.

The scripts select CUDA when available and otherwise run on CPU. The completed CSV summaries record CUDA for all Problem 1 runs and all Problem 2 runs. The final Problem 2 experiments were run with approved GPU access on an NVIDIA GeForce RTX 4070. Package versions used in the current environment were: `torch 2.12.0+cu130`, `torchvision 0.27.0+cu130`, `transformers 5.14.1`, `pandas 2.2.3`, `numpy 2.2.6`, and `matplotlib 3.10.9`.

Important status note: Problem 1 has 10-epoch results for all planned rows. Problem 2 has 5-epoch results for all planned rows. Every submitted summary row contains measured result values.

Reproducibility and Metric Details

CIFAR-100 contains 50,000 training images and 10,000 test images across 100 classes. The scripts train on the official training split and evaluate on the official test split. The CSV column names use `val_loss` and `val_accuracy_pct`, but these values are computed on the CIFAR-100 test split because no separate validation split was created for this homework. Randomness is controlled with seed 4106 for Python and PyTorch.

All models use cross-entropy loss and top-1 accuracy. Training time is measured as average wall-clock seconds per epoch, including evaluation at the end of each epoch. The history CSV files in the two results folders contain the per-epoch losses, accuracies, and runtime values used for the plots. The main commands used to reproduce the artifacts are:

```
python src/hw5_vit_resnet.py --run-all-vit --run-resnet
```

```
python src/hw5_swin.py --run-pretrained swin_tiny_pretrained
python src/hw5_swin.py --run-pretrained swin_small_pretrained
python src/hw5_swin.py --run-scratch --epochs 5 --batch-size 32 \
    --image-size 224
python tools/make_hw5_plots.py
```

Problem 1: Vision Transformer from Scratch vs. ResNet-18

Objective

The goal of Problem 1 is to compare several Vision Transformer configurations against a ResNet-18 baseline on CIFAR-100 and discuss accuracy, parameter count, estimated FLOPs, and training time.

Configurations

Model	Patch	Embed	Depth	Heads	MLP	Epochs	Batch	LR
ViT-A	4	256	4	4	1024	10	64	0.001
ViT-B	4	512	8	8	2048	10	64	0.001
ViT-C	8	256	4	4	1024	10	64	0.001
ViT-D	8	512	8	8	2048	10	64	0.001
ResNet-18	–	–	–	–	–	10	64	0.001

Implementation Summary

The custom ViT starts with a convolutional patch embedding layer whose kernel size and stride match the patch size. The flattened patch sequence is prepended with a learned class token, then combined with a learned positional embedding. The encoder is built from `nn.TransformerEncoderLayer` using GELU activations, multi-head self-attention, feed-forward MLP blocks, dropout, and layer normalization. The classifier uses the final normalized class-token representation and a linear head for 100 CIFAR-100 classes.

The ResNet-18 baseline uses `torchvision.models.resnet18(weights=None)`. Its first convolution is changed to a 3 by 3 stride-1 convolution and the initial max-pooling layer is removed so the model is better matched to 32 by 32 CIFAR images. All models use cross-entropy loss and Adam optimization. Evaluation runs without gradients on the CIFAR-100 test split and records validation/test loss and top-1 accuracy.

Complexity Calculation Details

For each ViT, the number of tokens is $N = (32/\text{patch_size})^2 + 1$, where the extra token is the class token. Patch embedding parameters are $(\text{patch_size}^2 \times 3 \times \text{embed_dim}) + \text{embed_dim}$. The learned class token and positional embedding add $\text{embed_dim} + N \times \text{embed_dim}$ parameters. Transformer and classifier parameters are counted directly from the PyTorch model.

ViT FLOPs are estimated per forward pass with the transformer approximation $\text{FLOPs}_{\text{layer}} \approx 4Nd^2 + 2N^2d + 2Ndm$, where d is embedding dimension and m is the MLP hidden dimension. The reported ViT FLOPs multiply that layer cost by the transformer depth and add the classifier head cost $d \times 100$. ResNet-18 FLOPs are estimated by summing convolution and linear multiply-adds from the actual CIFAR-adapted network, using the 3 by 3 stride-1 stem and removed max pool.

Results

Model	Params	FLOPs	Time/Epoch	Val Loss	Test Acc.	Status
ViT-A	3,214,692	2.13e8	23.01	3.2834	20.01%	10 epochs, CUDA
ViT-B	25,330,276	1.67e9	106.72	4.3232	4.51%	10 epochs, CUDA
ViT-C	3,239,268	5.41e7	12.22	3.8851	9.68%	10 epochs, CUDA
ViT-D	25,379,428	4.30e8	42.30	4.2694	4.95%	10 epochs, CUDA
ResNet-18	11,220,132	5.55e8	61.24	1.3919	61.20%	10 epochs, CUDA

Analysis

Among the ViT models, ViT-A performed best with 20.01% test accuracy and a final validation loss of 3.2834. ViT-C was the next best ViT at 9.68% while using the lowest estimated FLOP count. ViT-B and ViT-D were much larger, about 25.3 million trainable parameters each, but performed poorly in this 10-epoch run. The larger transformer settings appear harder to optimize from scratch on CIFAR-100 under this short schedule.

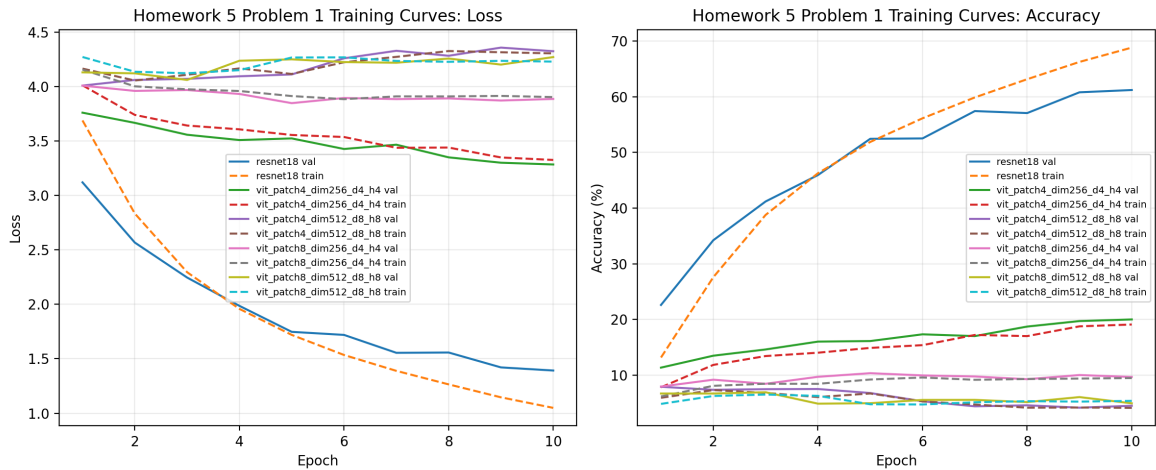
Patch size affected compute directly. With 32 by 32 inputs, patch size 4 creates 64 image tokens plus the class token, while patch size 8 creates 16 image tokens plus the class token. Since transformer attention scales strongly with sequence length, the patch size 8 configurations had much lower estimated FLOPs and lower runtime. Comparing ViT-A and ViT-C isolates this effect: both use embedding dimension 256, depth 4, and 4 heads, but the 8 by 8 patch model lowers FLOPs from 2.13e8 to 5.41e7 while accuracy drops from 20.01% to 9.68%. The larger patch gives faster training but loses fine spatial detail.

Increasing model capacity did not help under the short from-scratch schedule. ViT-B has the same 4 by 4 patch size as ViT-A but increases embedding dimension, depth, heads, and MLP width; parameters rise from 3.21 million to 25.33 million and FLOPs rise from 2.13e8 to 1.67e9, while accuracy falls from 20.01% to 4.51%. The same pattern appears between ViT-C and ViT-D. The larger models have enough capacity, but the training curves show they do not optimize well in only 10 epochs with this simple Adam setup.

The ResNet-18 baseline was strongest overall, reaching 61.20% test accuracy after 10 epochs. It used more parameters and FLOPs than ViT-A and ViT-C, but far less compute than the largest ViT-B run and still produced much higher accuracy. This result is expected on CIFAR-100 because convolutional inductive bias helps ResNet learn local image features efficiently, while a from-scratch ViT generally needs more data, longer schedules, stronger regularization, or pretraining to become competitive.

The training-curve figure verifies model behavior beyond final accuracy. ResNet-18 shows steadily decreasing training and test loss, so its accuracy improvement is backed by a real learning curve. ViT-A and ViT-C also show some loss reduction, but the deeper/wider ViT-B and ViT-D curves remain close to high cross-entropy loss and low accuracy, confirming that their poor final results are training behavior rather than a reporting artifact.

The plot below shows the per-epoch loss and accuracy curves for each Problem 1 run.



Problem 2: Fine-Tuning Pretrained Swin Transformers vs. Training from Scratch

Objective

The goal of Problem 2 is to compare pretrained Swin-Tiny and Swin-Small head-only fine-tuning against a scratch Swin-style model on CIFAR-100.

Implementation Summary

The pretrained Swin models are loaded from Hugging Face using `SwinForImageClassification` and its `from_pretrained()` method with `microsoft/swin-tiny-patch4-window7-224` and `microsoft/swin-small-patch4-window7-224`. The classifier head is replaced for 100 CIFAR-100 classes with `ignore_mismatched_sizes=True`. For the pretrained experiments, all backbone parameters are frozen and only classifier parameters remain trainable. The script uses Adam, cross-entropy loss, batch size 32, learning rate $2e-5$, and 224 by 224 resized CIFAR-100 images.

The scratch model uses `torchvision.models.swin_t(weights=None)` with its head replaced for 100 classes. Unlike the pretrained runs, all scratch-model parameters are trainable. The scratch run uses 224 by 224 resized CIFAR-100 inputs, batch size 32, learning rate 0.001, and 5 epochs on CUDA.

Model Size Details

For the frozen pretrained Swin models, total parameters count the full backbone plus classifier. Trainable parameters count only tensors with `requires_grad=True`. The replacement classifier has $768 \times 100 + 100 = 76,900$ trainable parameters for both Swin-Tiny and Swin-Small. The scratch Swin-Tiny-sized model has the same total parameter count as Swin-Tiny, but all parameters are trainable because it is initialized from scratch.

Results

Model	Pretrained	Frozen	Total Params	Trainable Params	Time/Epoch	Val Loss	Test Acc.	Status
Swin-Tiny	Yes	Yes	27,596,254	76,900	166.32	1.7003	64.78%	5 epochs, CUDA
Swin-Small	Yes	Yes	48,914,158	76,900	254.73	1.4918	68.74%	5 epochs, CUDA
Scratch Swin	No	No	27,596,254	27,596,254	656.55	4.6087	1.00%	5 epochs, CUDA

Analysis

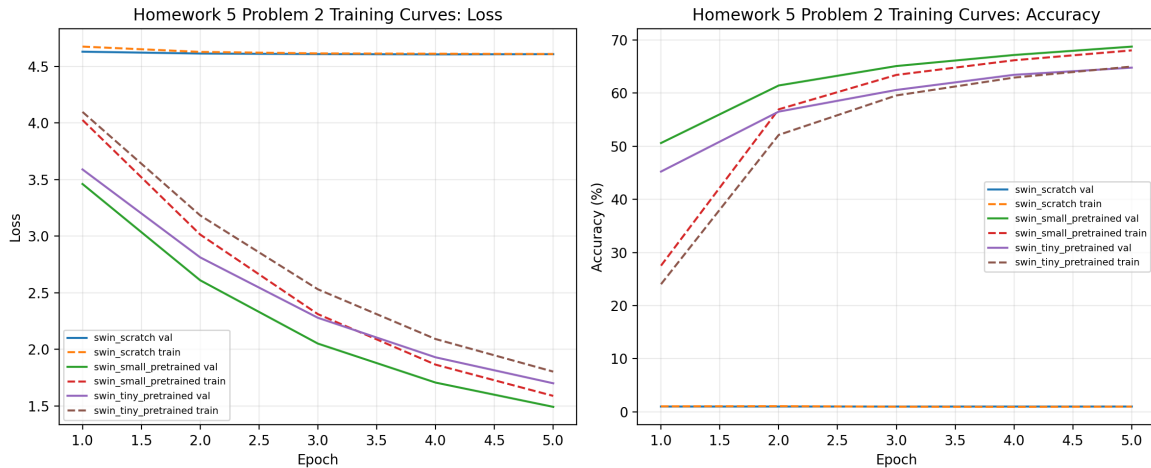
The completed pretrained runs show a clear transfer-learning advantage after five epochs. Swin-Small reached 68.74% test accuracy, and Swin-Tiny reached 64.78%. These accuracies are much higher than the from-scratch transformer results because the Swin backbones already contain useful image features from pretraining.

Swin-Small performed better than Swin-Tiny in the completed artifacts, with lower validation loss and higher accuracy. The trainable parameter count is the same for both pretrained rows because the backbone is frozen and only the classifier head is trained. The total parameter count is higher for Swin-Small, so it has a larger memory footprint and higher time per epoch even though the optimizer updates the same 76,900 classifier parameters.

The scratch Swin baseline stayed near chance accuracy, reaching 1.00% after five CUDA epochs. This is well below both pretrained Swin rows even though the scratch model has the same total parameter count as Swin-Tiny and updates all 27.6 million parameters. The result supports the expected advantage of transfer learning: with a short five-epoch schedule, the randomly initialized Swin model did not learn useful CIFAR-100 features. A stronger from-scratch transformer result would likely require a longer schedule, warmup, tuned learning-rate decay, stronger augmentation, and additional regularization.

The Problem 2 training curves show why the final table differs across models. The pretrained Swin losses decrease each epoch and validation accuracy rises into the 60% range, which indicates the classifier head is fitting useful frozen features. Scratch Swin remains near cross-entropy loss $\ln(100) \approx 4.605$ and about 1% accuracy, which is chance performance for 100 classes. This confirms that the scratch model did not train meaningfully in the allotted five epochs.

The plot below shows the per-epoch loss and accuracy curves for each Problem 2 run.



Conclusion

The results support two main takeaways. First, ResNet-18 substantially outperformed all from-scratch ViT configurations after 10 epochs on CIFAR-100, and ViT-A was the best transformer configuration among the four tested. Second, pretrained Swin feature extractors were much stronger than scratch Swin under the same five-epoch Problem 2 schedule, showing the practical value of transfer learning when compute and training time are limited.